

INDIA RUNS • DATA & AI CHALLENGE

Intelligent Candidate Discovery & Ranking

Hybrid offline ranker — semantic fit, trap-aware, production-scale

Team Name
Schadn

Team Leader
Rahul Kumar Singh

Problem Statement: Build an AI system that ranks 100K candidates the way a recruiter would — understanding role fit from career context, skills, and behavioral signals, not keyword matching.

Solution Overview

What we built and why it beats keyword ATS

Proposed Solution

RecruitGPT X offline ranker: scores 100,000 candidates on CPU in ~49–60 seconds.

Acts as an AI recruiter — reads title, career narrative, IR skills, availability, and engagement.

Outputs a top-100 shortlist with calibrated scores and 2-sentence grounded reasoning per row.

Pre-computed MiniLM bi-encoder + hybrid heuristics — no LLM calls at rank time.

Differentiation vs Traditional ATS

Keyword ATS: string match → misses semantic fit, ranks honeypot profiles high.

Our ranker: career semantics + structural honeypot traps + template-blurb dedup.

Availability gate: not open-to-work cannot reach top-10.

Explainable: every rank cites real profile facts — no hallucinated employers or skills.

Production constraint: CPU-only, offline — scales to 200K+ without per-candidate API cost.

JD Understanding & Candidate Signals

Parsed from job_description.docx → jd_config.py

Key JD Requirements

Role: Senior AI Engineer — retrieval, ranking, embeddings, vector DBs in production.

Experience: 5–9 years ideal; penalize thin tenure and over-senior outliers.

Location: India-based; Pune/Noida preferred in scoring.

Production language: shipped systems, not research-only or framework demos.

Behavioral modifier from redrob_signals_doc: response rate, recruiter saves, engagement.

Candidate Signals We Evaluate

Title alignment: Rec Sys Engineer, Search Engineer, Senior ML/NLP Engineer.

Core IR skills: FAISS, Pinecone, Elasticsearch, OpenSearch, Weaviate, retrieval pipelines.

Career semantic text: production stories in role descriptions (not keyword stuffing).

Skill trust: endorsements, months-used, verified badges — noise skills penalized.

Availability: open_to_work, notice period ($\leq 30d$ boosted, 90–120d penalized).

Engagement: profile views, recruiter saves, interview completion rate.

Trap signals: weak title + inflated AI skills, impossible tenure, recycled blurbs.

Ranking Methodology

Hybrid v6 scorer — retrieve, score, rank, explain

Pipeline Steps

1. Load candidates.jsonl → build CandidateIndex (title, skills, career, signals).
2. Score each candidate with weighted hybrid components + modifiers.
3. Apply honeypot penalty, availability gate, notice modifier, blurb dedup.
4. Calibrate raw scores → sort descending → assign ranks 1–

Models & Algorithms

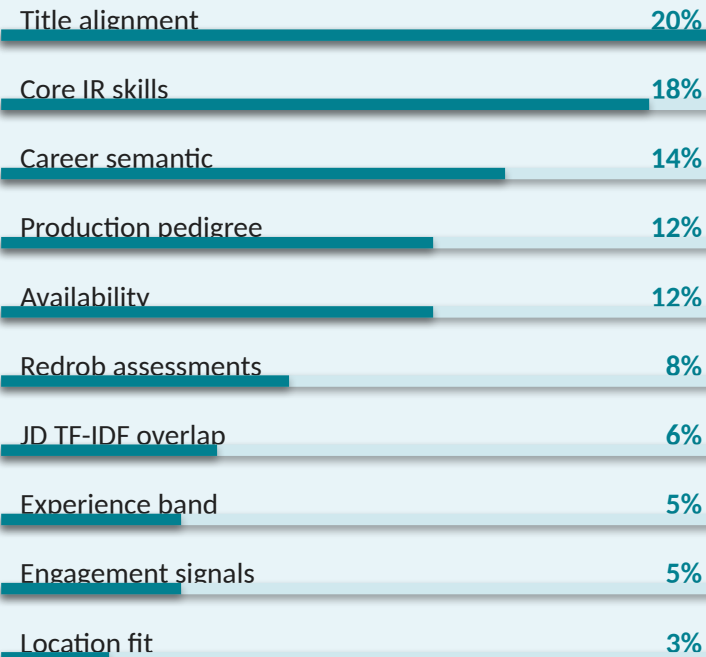
Bi-encoder: all-MiniLM-L6-v2 (committed embeddings.fp16.npz, 100K vectors).

TF-IDF JD overlap for lexical alignment with parsed JD.

Heuristic modifiers: consulting penalty, research-only cap, FAANG-current note.

Cross-encoder: OFF (RANKER_USE_CROSS_ENCODER=0) for reproducibility.

v6 Signal Weights (DEFAULT_WEIGHTS)



Explainability & Data Validation

Grounded reasoning + trap defense

How Rankings Are Explained

reasoning column: 2 sentences per candidate in submission.csv.

Sentence 1: specific facts — title, employer, years, location, named IR skills.

Sentence 2: JD connection + honest concerns (notice period, thin IR depth, big-tech risk).

Built from score components — never from an LLM → no hallucinated skills or employers.

Stage 4 mock review: 10/10 sampled rows pass all 6 quality checks.

100% unique reasoning strings; no empty or copy-paste templates.

Handling Bad or Suspicious Profiles

Structural honeypots: impossible tenure vs company founding date, expert skills at 0 months.

Title-skill mismatch: HR Manager / Accountant with inflated AI skill lists → heavy demotion.

Template-blurb penalty: recycled career descriptions shared across thousands of profiles.

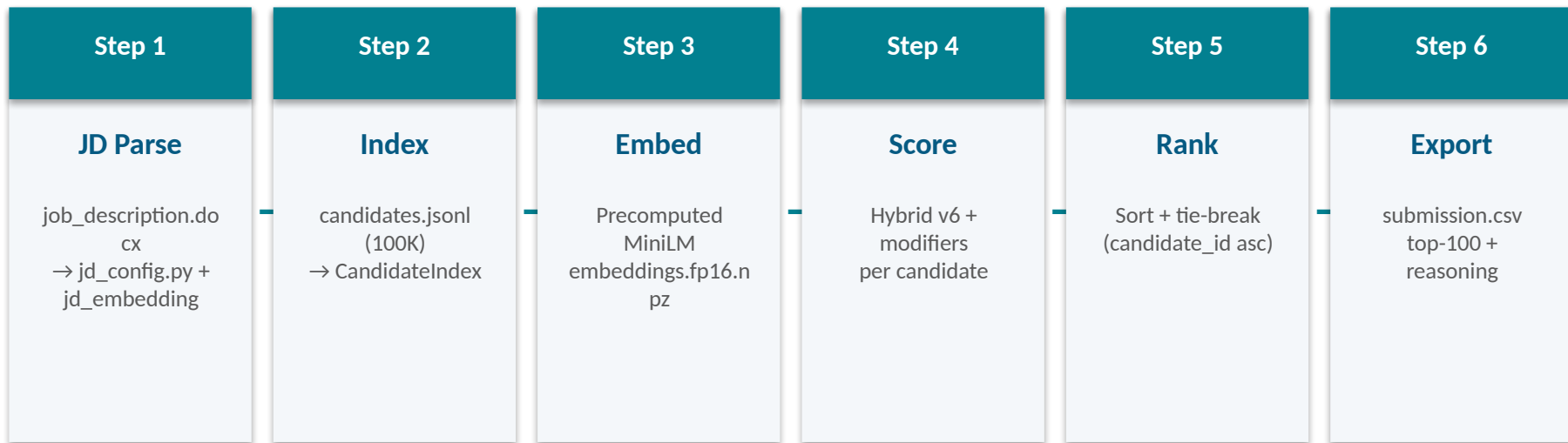
Noise skills penalized: HTML, Tailwind, generic frameworks without IR depth.

Availability hard gate: open_to_work=false capped out of top-10.

Result: 0 structural honeypots in our top-100 submission (0% vs 10% disqualify threshold).

End-to-End Workflow

JD input → ranked CSV output



Reproduce: `./scripts/reproduce_ranking.sh · python rank.py --candidates ./data/candidates.jsonl --out ./submission.csv`
Byte-verified: `scripts/verify_submission_artifact.py` confirms submission.csv matches fresh rank.py output.

System Architecture

Submission ranker (graded) vs demo platform (optional)

GRADED PATH — produces submission.csv

rank.py

redrob_ranker.py

features.py

embeddings.py

honeypot.py

availability.py

+ career_blurb.py (template dedup) · assessment.py (IR skill tests) · rerank.py (CE off)

Input: candidates.jsonl + embeddings.fp16.npz → Output: submission.csv (100 rows)

DEMO PATH — recruiter UI (not graded)

frontend/ (Next.js 14) → backend/ (FastAPI + LangGraph 7-agent pipeline) → explainability, chat, analytics

Uses top-100 import from submission.csv for dashboard — does NOT produce the graded artifact.

Results & Performance

Top picks, validation, compute compliance

Top-5 Ranked Candidates

- #1 CAND_0002025 · 0.9900 · Senior AI Engineer @ Apple
- #2 CAND_0046064 · 0.9324 · Senior NLP Engineer @ Salesforce
- #3 CAND_0010685 · 0.8659 · NLP Engineer @ Rephrase.ai
- #4 CAND_0055905 · 0.8510 · Sr ML Engineer @ Flipkart
- #5 CAND_0018499 · 0.8041 · Sr ML Engineer @ Zomato

Validation Results

100 rows · ranks 1–100 unique · scores monotonic · 0 honeypots.

All 100 candidate_ids verified in official candidates.jsonl.

Byte-reproducible artifact (verify_submission_artifact.py: PASS).

Reasoning: 100% unique, Stage 4 mock 10/10 rows pass.

~49–60s

on 100K candidates

CPU only

no GPU · no network

16 GB

RAM limit met

Well under 5-minute wall-clock limit · embeddings bundle ~68MB fp16 · Docker reproduction supported

Technologies Used

Chosen for reproducibility, scale, and spec compliance

Submission Ranker (Graded)

Python 3.11 · NumPy · PyYAML — minimal deps, no torch at rank time.

all-MiniLM-L6-v2 bi-encoder — precomputed, committed as embeddings.fp16.npz.

TF-IDF + lexical matchers for JD overlap and skill phrase detection.

Docker: docker-compose.ranker.yml for Stage 3 sandbox reproduction.

pytest: challenge/test_ranker.py (31 tests passing).

AI dev tools: Cursor + Grok (architecture/review only — no candidate data to LLMs).

Demo Platform (Optional)

Next.js 14 + Tailwind — recruiter dashboard, analytics, candidate actions.

FastAPI + LangGraph — 7-agent pipeline for interactive ranking explanations.

PostgreSQL / SQLite — workspace data; top-100 import from submission.csv.

HuggingFace Spaces — sandbox for sample ranking (≤ 100 candidates).

Vercel + Render — live product demo at recruitgpt-x.vercel.app.

Submission Assets

Everything judges need to reproduce and review

GitHub Repository

<https://github.com/rahulx2001/recruitgpt-x>

Full source + README + reproduce script

Ranked CSV

[submission.csv \(top-100\)](#)

candidate_id, rank, score, reasoning

HF Sandbox

huggingface.co/spaces/rahulsinghx2001/recruitgpt-ranker

Sample ranking on ≤ 100 candidates, CPU

Live Demo

recruitgpt-x.vercel.app

Recruiter UI — dashboard, analytics, AI chat

Reproduce

[./scripts/reproduce_ranking.sh](#)

Single command → byte-verified submission.csv

Metadata

[submission_metadata.yaml](#)

Team Schadn · Rahul Kumar Singh · compute + AI declaration

Contact: rahulsinghx2001@gmail.com · +91-8539816642 · Team Schadn · Rahul Kumar Singh

Thank You

Schadn · Rahul Kumar Singh

RecruitGPT X — Ranking candidates like a great recruiter, not a keyword filter.

github.com/rahulx2001/recruitgpt-x